PROJETO

ARENA TECH: PLANEJAMENTO E VIABILIDADE DA MARATONA GAMER

Um programa em C para calcular custos, analisar recursos e decidir sobre a realização de um evento

MODALIDADE	ENTREGA	FORMATO
Duplas ou trios	Desenvolvimento em aula + avaliação por pares	Código-fonte em C + relatório de testes e revisão

1. Contexto do desafio

A turma foi convidada a organizar a Arena Tech, uma pequena maratona gamer no campus. Antes de confirmar o evento, a equipe precisa estimar a quantidade de times, o consumo de energia, os custos envolvidos e verificar se a infraestrutura disponível é suficiente. Além de realizar os cálculos, o programa deverá analisar os resultados e emitir um diagnóstico sobre a viabilidade do evento.

2. Objetivos do projeto

- transformar um problema contextualizado em uma solução computacional;
- planejar a solução antes da implementação;
- aplicar variáveis, entrada e saída, operadores e estruturas de decisão;
- testar diferentes caminhos de execução do programa;
- explicar as decisões implementadas no código;
- avaliar, por meio da revisão por pares, a solução desenvolvida por outro grupo.

3. Conteúdos mobilizados

- estrutura básica de um programa em C e uso de #include <stdio.h>;
- declaração e utilização de variáveis dos tipos int e float;
- entrada e saída de dados com scanf() e printf();
- operadores aritméticos, atribuição, parênteses e precedência;
- divisão real, conversão de tipos e formatação com duas casas decimais;
- operadores relacionais e operadores lógicos (&&, || e !);
- estruturas condicionais if, if...else e if...else if...else;
- uso opcional da biblioteca math.h e da função ceil().

4. Dados que o programa deverá solicitor (Entrada de dados esperada)

Variável sugerida	Informação	Tipo	Exemplo
qte_participantes	Quantidade total de participantes	int	30
qte_jogadores_por_time	Quantidade de jogadores em cada time	int	5
qte_computadores	Quantidade de computadores disponíveis	int	30
potencia	Potência média de cada computador, em watts	float	800
duracao	Duração do evento, em horas	float	2

Variável sugerida	Informação	Tipo	Exemplo
preco_kwh	Preço de 1 kWh de energia	float	1.2
preco_kit	Preço de um kit de alimentação por participante	float	20000
outros_custos	Outros custos do evento	float	150000
orcamento	Orçamento máximo disponível para o evento	float	100000

5. Cálculos obrigatórios

Resultado	Expressão de referência
Quantidade de times	arredondar qte_participantes / qte_jogadores_por_time
Consumo de energia	(qte_computadores * potencia * duracao) / 1000
Custo da energia	consume_energia * preco_kwh
Custo da alimentação	qte_participantes * preco_kit
Custo total	custo_energia + custo_alimentacao + outros_custos
Custo por participante	custo_total / qte_participantes
Saldo do orçamento	orcamento – custo_total

Dica: para calcular times incompletos, pode-se usar `ceil()` e `#include <math.h>`. No Linux/Codespaces, a compilação poderá exigir: gcc projeto1.c -o projeto1 -lm

6. Decisões obrigatórias do programa

Os cálculos deverão alimentar decisões que analisem o planejamento do evento. O programa deverá implementar as regras abaixo.

Análise	Regra	Resultado esperado
Infraestrutura	computadores >= participantes	INFRAESTRUTURA SUFICIENTE
Infraestrutura	computadores < participantes	INFRAESTRUTURA INSUFICIENTE + quantidade faltante
Consumo	consumoEnergia <= 20	BAIXO
Consumo	20 < consumoEnergia <= 40	MODERADO
Consumo	consumoEnergia > 40	ALTO

Para o orçamento, considere a margem de segurança de 5% do valor disponível:

- se custoTotal > orcamento: ACIMA DO ORÇAMENTO;
- se custoTotal <= orcamento e o saldo for menor ou igual a 5% do orçamento: NO LIMITE DO ORÇAMENTO;
- caso contrário: DENTRO DO ORÇAMENTO.

Decisão final do evento:

- NÃO RECOMENDADO: se a infraestrutura for insuficiente OU se o custo total ultrapassar o orçamento;
- APROVADO COM RESSALVAS: se a infraestrutura for suficiente, o orçamento não for ultrapassado e o consumo de energia for ALTO;
- APROVADO: nos demais casos em que a infraestrutura seja suficiente e o orçamento não seja ultrapassado.

7. Saída esperada

Ao final, o programa deverá exibir um relatório legível. O formato abaixo é apenas uma referência; pequenas variações de apresentação são permitidas.

```
============== ARENA TECH ==============
Participantes: 30
Times necessários: 6
Computadores disponíveis: 30
Infraestrutura: SUFICIENTE

Consumo estimado: 48.00 kWh
Classificação do consumo: ALTO
Custo da energia: R$ 57.60
Custo da alimentação: R$ 600.00
Outros custos: R$ 150.00
CUSTO TOTAL: R$ 807.60
CUSTO POR PARTICIPANTE: R$ 26.92

Orçamento disponível: R$ 1000.00
Saldo: R$ 192.40
Situação do orçamento: DENTRO DO ORÇAMENTO

DECISÃO FINAL: APROVADO COM RESSALVAS
Motivo: consumo elevado de energia.
========================================
```

8. Teste mínimo obrigatório

Antes da avaliação por pares, execute o programa utilizando pelo menos os dados abaixo. O objetivo é verificar cálculos e um dos caminhos das estruturas de decisão.

Participantes	Jog./time	PCs	Potência	Horas	R$/kWh	Kit	Outros	Orçamento
30	5	30	800 W	2 h	R$ 1,20	R$ 20,00	R$ 150,00	R$ 1.000,00

Para esse conjunto de dados, o programa deverá obter, entre outros resultados:

- consumo de energia = 48,00 kWh;
- classificação do consumo = ALTO;
- custo total = R$ 807,60;
- infraestrutura = SUFICIENTE;
- situação do orçamento = DENTRO DO ORÇAMENTO;
- decisão final = APROVADO COM RESSALVAS.

9. Avaliação por pares: teste o projeto dos colegas

Na segunda parte da aula, um integrante permanecerá na bancada e os demais visitarão outro grupo. O grupo visitante atuará como equipe de revisão.

O grupo avaliador deverá executar o programa com diferentes entradas e registrar, para cada teste:

Entrada → Resultado esperado → Resultado obtido

Os testes deverão procurar ativar diferentes caminhos das decisões. No mínimo, tente produzir:

- um caso com infraestrutura suficiente;
- um caso com infraestrutura insuficiente;

- um caso de consumo BAIXO, um MODERADO e um ALTO;
- um caso DENTRO DO ORÇAMENTO, um NO LIMITE e um ACIMA DO ORÇAMENTO;
- pelo menos dois resultados diferentes para a decisão final.

Pergunta norteadora: o programa funciona apenas para o exemplo apresentado ou se comporta corretamente em diferentes situações?

10. Registro da avaliação por pares

O relatório de avaliação deverá ser preenchido pelo grupo visitante e entregue conforme orientação do professor. Ele deverá conter:

- identificação do grupo avaliado e dos avaliadores;
- pontos fortes do código e da solução;
- pontos frágeis ou oportunidades de melhoria;
- registro dos testes realizados e dos resultados obtidos;
- observações sobre clareza, organização e funcionamento do código;
- indicação de pelo menos uma melhoria recomendada.

11. Uso de Inteligência Artificial

O uso de IA é permitido como apoio ao processo de aprendizagem. Caso o grupo utilize alguma ferramenta, deverá declarar o uso e ser capaz de explicar o que foi feito.

No relatório e durante a apresentação, o grupo deverá responder:

- Houve uso de IA? Qual ferramenta?
- Em qual etapa a IA foi utilizada: compreensão do problema, planejamento, escrita do código, depuração ou outra?
- Que parte da solução recebeu auxílio da IA?
- O grupo consegue explicar o código produzido ou modificado com auxílio da IA?
- O uso da IA contribuiu para a aprendizagem? Como?
- Antes do projeto, o grupo conseguiria resolver um problema semelhante sem IA? E depois da atividade?

Importante: a avaliação não será baseada em “detectar IA”, mas na compreensão da solução, na capacidade de explicar o código e no uso consciente da ferramenta.

12. Retorno ao grupo e melhoria da solução

Depois da avaliação, todos retornam ao grupo original. O grupo deverá analisar o feedback recebido e:

- selecionar pelo menos uma sugestão de melhoria;
- ajustar o código ou a apresentação do projeto;
- registrar qual melhoria foi realizada;
- caso alguma sugestão não seja implementada, justificar a decisão;
- registrar brevemente o que aprendeu ao analisar e receber a análise de outro projeto.

13. O que deverá ser entregue

- arquivo projeto1_arena_tech.c, devidamente compilado e testado (**TODOS OS ALUNOS DEVEM ENTREGAR**);
- registro da execução do teste mínimo obrigatória (**Print da tela – somente saída**);
- relatório de avaliação por pares preenchido;

- README.md curto, contendo nomes e RGMs dos integrantes, descrição da solução, registro de uso de IA e melhoria realizada após o feedback (**PODE UTILIZAR IA – SEM PROBLEMAS**);

14. Regras importantes

- O projeto deverá ser realizado em duplas ou trios.
- Todos os integrantes devem compreender o programa e conseguir explicar os cálculos e as estruturas de decisão utilizadas.
- Utilize nomes de variáveis claros, indentação adequada e mensagens que orientem o usuário durante a execução.
- As estruturas condicionais fazem parte dos requisitos obrigatórios deste projeto.
- Não serão aceitos programas que apenas exibam resultados fixos, sem leitura de dados pelo teclado.
- O programa deverá ser testado com diferentes conjuntos de dados antes da avaliação por pares.
- A utilização de IA é permitida como apoio, mas o grupo deverá revisar, testar e compreender todo o código entregue.
- Durante a avaliação por pares, o objetivo é ajudar outra equipe a melhorar a solução, e não apenas procurar erros.

15. Critérios de avaliação

Conforme estabelecido em aula, o projeto será avaliado da seguinte forma:

Critério	Descrição	Valor
Entrada e saída	O programa compila, executa, realiza os cálculos corretamente e apresenta saídas coerentes.	0,25
Estruturas de decisão	Uso correto de operadores relacionais/lógicos e das estruturas if, if...else e if...else if...else.	0,25

16. Desafios extras (opcionais)

- usar switch...case para permitir que o usuário escolha uma categoria de estrutura do evento (Econômica, Padrão ou Premium), acrescentando um custo específico ao planejamento;
- validar uma opção inválida no switch com default;
- informar o percentual do orçamento já comprometido pelo evento;
- criar mensagens mais detalhadas para explicar os motivos da decisão final;
- personalizar o relatório final com nome, data e identidade visual da Arena Tech.

Desenvolver → Testar → Explicar → Avaliar → Melhorar

Programar não é apenas escrever código: é construir, testar, explicar, revisar e melhorar soluções.